GitLab CI/CD — основы

Что такое GitLab, зачем он нужен и как писать хорошие pipelines

Author: Aliaksandr Tsimokhau

Agenda

Part 1: Что такое GitLab и зачем

Платформа, GitLab CI/CD и анатомия pipeline

Part 2: Пишем `.gitlab-ci.yml`

Stages, jobs, зависимости, переменные, артефакты

Part 3: Собираем pipeline по шагам

От первого job до полного DAG-пайплайна

Part 4: Когда запускать jobs — `rules`

Условия: changes, if, exists

Part 1: Что такое GitLab и зачем

Единая DevOps-платформа: code → CI/CD → deploy

GitLab — больше, чем «git + CI»

DevSecOps platform: одна установка покрывает весь SDLC.

10 этапов SDLC в одной платформе:

- Manage · Plan · Create
- **Verify** (CI) · Package · Secure
- **Release** (CD) · Configure
- Monitor · Govern

Что входит «из коробки»:

- Git hosting (Gitaly)
- Issues, MR, wiki
- CI/CD + Runners
- Container & Package Registry
- Security scanners (SAST/DAST/SCA)
- Pages · Environments · Feature flags

> 💡 Конкуренты — это **набор интегрированных tools**: GitHub + Actions + Dependabot + ghcr + Pages. У GitLab всё это — один продукт.

What Is GitLab CI/CD?

- **GitLab CI/CD** — инструмент для continuous практик: CI · Delivery · Deployment
- Раннее обнаружение ошибок в SDLC
- Код в проде соответствует compliance и coding standards

<svg viewBox="0 0 480 250" width="64%" style="max-height:380px" xmlns="http://www.w3.org/2000/svg" font-family="sans-serif"> <text x="8" y="36" font-size="16" font-weight="bold" fill="#1d4ed8">CI</text> <rect x="55" y="14" width="190" height="34" rx="8" fill="#2d5a8e"/> <text x="150" y="36" text-anchor="middle" font-size="13" fill="#fff">commit → build → test</text> <text x="8" y="104" font-size="16" font-weight="bold" fill="#6d28d9">CD</text> <rect x="55" y="78" width="320" height="34" rx="8" fill="#7c3aed"/> <text x="215" y="100" text-anchor="middle" font-size="12.5" fill="#fff">... → staging → 👤 manual gate → prod</text> <text x="55" y="132" font-size="12" fill="#6d28d9">Continuous Delivery — релиз по кнопке</text> <text x="8" y="194" font-size="16" font-weight="bold" fill="#047857">CD</text> <rect x="55" y="168" width="415" height="34" rx="8" fill="#047857"/> <text x="262" y="190" text-anchor="middle" font-size="12.5" fill="#fff">... → prod (автоматически, без человека)</text> <text x="55" y="222" font-size="12" fill="#047857">Continuous Deployment — каждый green pipeline → prod</text> </svg>

GitLab CI pipeline



Пример CI/CD pipeline для GitLab

```yaml
stages:
  - build
  - test
  - deploy

build:
  stage: build
  script:
    - docker build -t my-app:latest .
    - docker tag my-app:latest registry.example.com/my-app:latest
    - docker push registry.example.com/my-app:latest

test:
  stage: test
  script:
    - docker run --rm my-app:latest pytest

deploy:
  stage: deploy
  script:
    - kubectl apply -f k8s/deployment.yaml
  environment:
    name: production
    url: https://example.com
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

Part 2: Пишем `.gitlab-ci.yml`

Синтаксис pipeline с примерами

Создание файла `.gitlab-ci.yml`

```yaml
stages:
  - build
  - test
  - deploy

variables:
  APP_NAME: "my-app"
  APP_VERSION: "1.0.0"

build:
  stage: build
  script:
    - echo "Building the project..."
    - ./build_script.sh
  artifacts:
    paths:
      - build/

test:
  stage: test
  script:
    - echo "Running tests..."
    - ./test_script.sh
  dependencies:
    - build
```

Стадии (stages)

Pipeline содержит **stages**, каждый stage содержит **jobs**

```yaml
stages:
  - build
  - test
  - deploy
```



Задачи (jobs) — сборка

Задача сборки выполняется на стадии `build` и включает команды для сборки проекта.

```yaml
build:
  stage: build
  script:
    - echo "Building the project..."
    - ./build_script.sh
  artifacts:
    paths:
      - build/
```

Задачи (jobs) — тестирование

Задача тестирования выполняется на стадии `test` и зависит от артефактов из `build`.

```yaml
test:
  stage: test
  script:
    - echo "Running tests..."
    - ./test_script.sh
  dependencies:
    - build
```

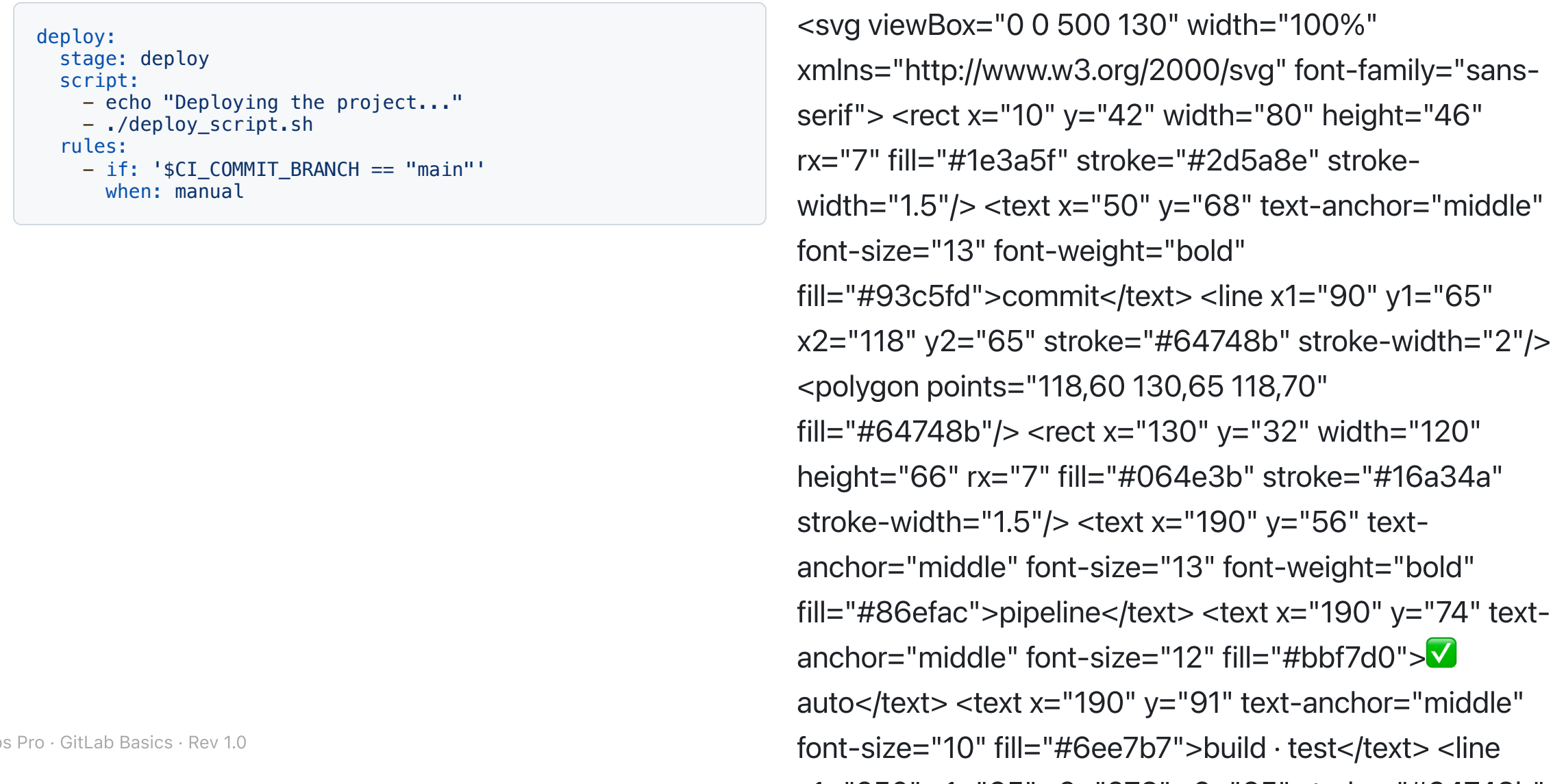
Зависимости (dependencies)

Зависимости позволяют задачам использовать артефакты, созданные в предыдущих задачах.

```
dependencies:
  - build
```

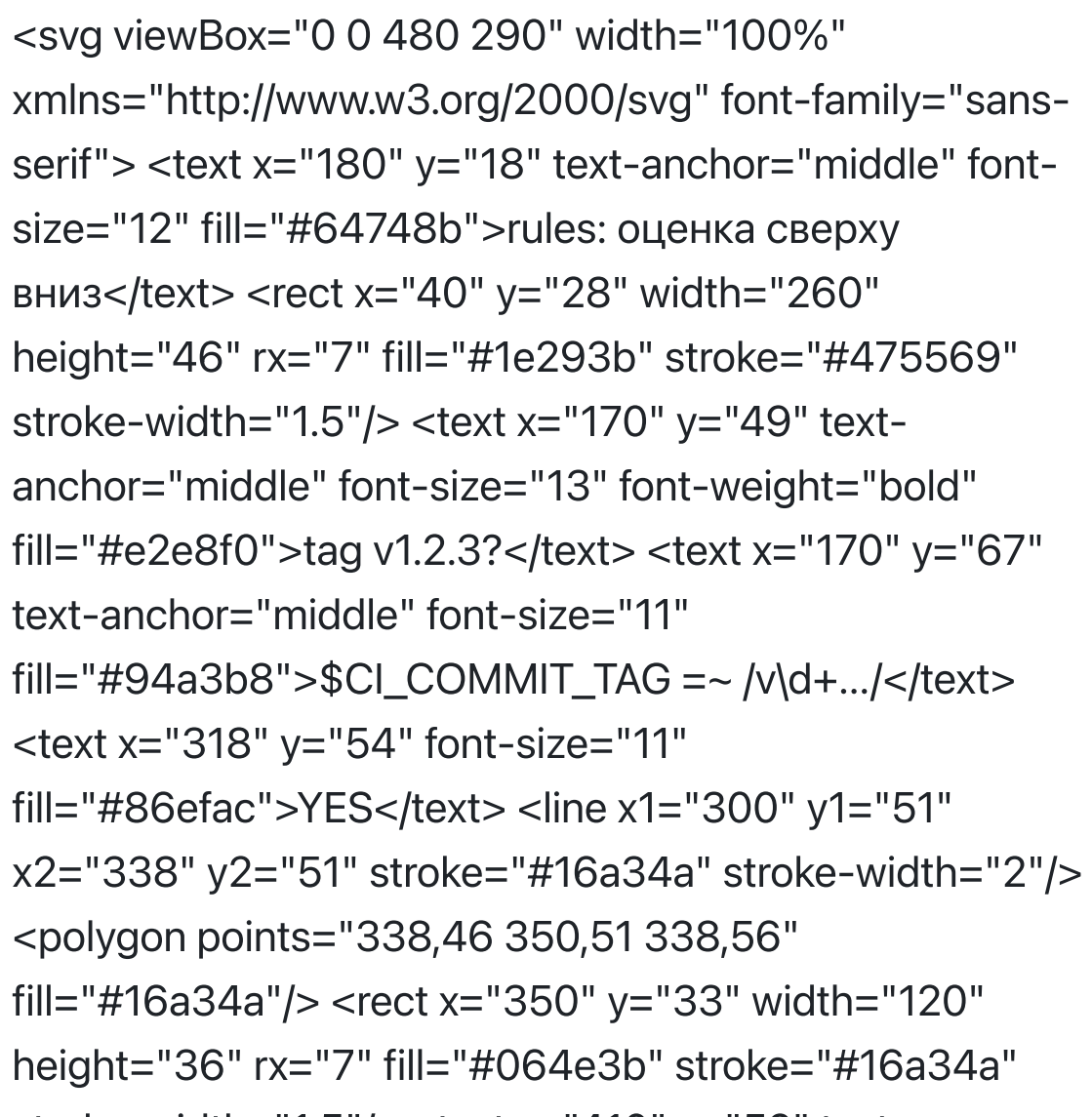
Задача развертывания (deploy)

```yaml
deploy:
  stage: deploy
  script:
    - echo "Deploying the project..."
    - ./deploy_script.sh
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: manual
```



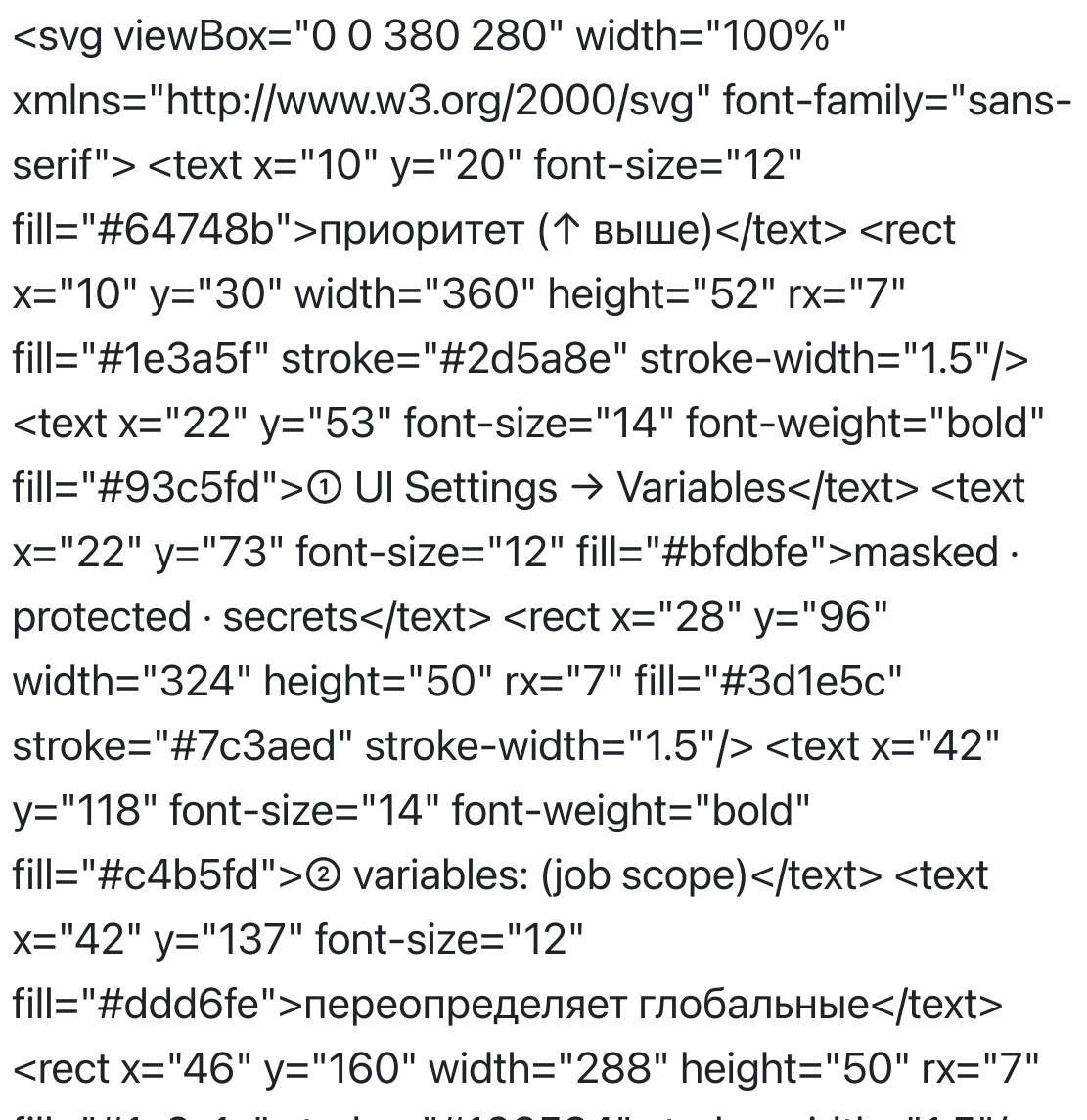
Триггеры (triggers)

```yaml
deploy:
  stage: deploy
  script:
    - echo "Deploying the project..."
    - ./deploy_script.sh
  rules:
    - if: '$CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/'
      when: on_success
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: manual
    - when: never
```



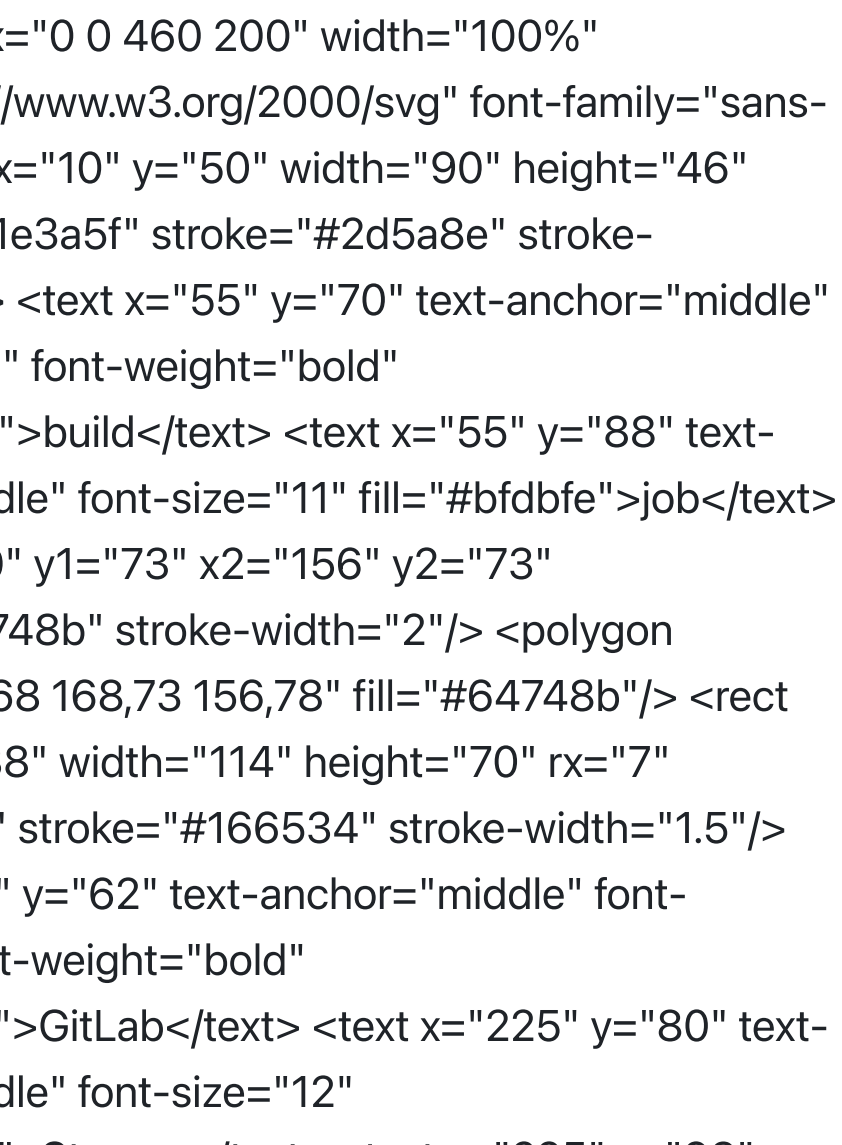
Переменные окружения

```yaml
variables:
  APP_NAME: "my-app"
  APP_VERSION: "1.0.0"
```



Артефакты

```yaml
artifacts:
  paths:
    - build/
  expire_in: 1 week
  reports:
    junit: report.xml
```



Part 3: Собираем pipeline по шагам

От первого job до полного DAG

Walkthrough — собираем pipeline по шагам

Цель: разобраться с моделью выполнения jobs — **sequentially / parallel / out of order** — через прогрессивный пример.

```
Шаг 1: Один test job — простейший .gitlab-ci.yml
Шаг 2: Артефакт — делаем сборку скачиваемой
Шаг 3: Stages — sequential execution (test → package)
Шаг 4: Compile stage — не дублируем работу
Шаг 5: Parallel — pack-gz и pack-iso в одном stage
Шаг 6: DAG через `needs` — out of order
```

> 💡 Каждый шаг — это маленькое улучшение. Так же работает adoption в реальной команде.

Шаг 1 — первый test job

```
test:
  before_script:
    - echo "Hello " | tr -d "\n" > file1.txt
    - echo "world" > file2.txt
  script: cat file1.txt file2.txt | grep -q 'Hello world'
```

Что происходит:

- GitLab Runner поднимает контейнер (по умолчанию `ruby:3.1`)
- выполняет `before_script` → `script`
- exit code 0 → job ✅ ; non-zero → job ❌

Шаг 2 — артефакты сборки

Артефакты делают результаты job'а **скачиваемыми и доступными для downstream jobs**:

```
test:
  script: cat file1.txt file2.txt | grep -q 'Hello world'

package:
  script: cat file1.txt file2.txt | gzip > packaged.gz
  artifacts:
    paths:
      - packaged.gz
```

> ⚠️ **Без `artifacts:` файл `packaged.gz` будет потерян** — workspace удаляется после job

Шаг 3 — sequential execution через `stages`

```yaml
stages:
  - test
  - package

test:
  stage: test
  script: cat file1.txt file2.txt | grep -q 'Hello world'

package:
  stage: package
  script: cat file1.txt file2.txt | gzip > packaged.gz
  artifacts:
    paths:
      - packaged.gz
```

Правило: все jobs в `package` ждут окончания **всех** jobs в `test`.

Шаг 4 — общий артефакт между jobs

Не повторяем работу: `compile` собирает один раз, `test` и `package` используют:

```yaml
stages:
  - compile
  - test
  - package

compile:
  stage: compile
  script: cat file1.txt file2.txt > compiled.txt
  artifacts:
    paths:
      - compiled.txt
    expire_in: 20 minutes

test:
  stage: test
  script: cat compiled.txt | grep -q 'Hello world'

package:
  stage: package
  script: cat compiled.txt | gzip > packaged.gz
  artifacts:
    paths:
      - packaged.gz
```

Шаг 5 — parallel jobs в одном stage

Два формата упаковки параллельно:

```
pack-gz:
  stage: package
  script: cat compiled.txt | gzip > packaged.gz
  artifacts:
    paths:
      - packaged.gz

pack-iso:
  stage: package
  before_script:
    - echo "ipv6" >> /etc/modules
    - apk update
    - apk add xorriso
  script:
    - mkisofs -o ./packaged.iso ./compiled.txt
  artifacts:
    paths:
      - packaged.iso
```

> 💡 Несколько jobs в одном stage → **параллельное выполнение** (ограничено количеством runners)

Шаг 6 — DAG: out-of-order через `needs`

```yaml
pack-gz:
  stage: package
  script: cat compiled.txt | gzip > packaged.gz
  needs: ["test"]
  artifacts:
    paths:
      - packaged.gz

pack-iso:
  stage: package
  before_script:
    - apk update && apk add xorriso
  script:
    - mkisofs -o ./packaged.iso ./compiled.txt
  needs: ["test"]
  artifacts:
    paths:
      - packaged.iso
```

`needs:` снимает stage-барьер — job стартует как только указанные dependencies закончились.



Полный pipeline — собираем всё вместе

```yaml
image: alpine

stages:
  - compile
  - test
  - package

compile:
  stage: compile
  before_script:
    - echo "Hello " | tr -d "\n" > file1.txt
    - echo "world" > file2.txt
  script: cat file1.txt file2.txt > compiled.txt
  artifacts:
    paths: [compiled.txt]
    expire_in: 20 minutes

test:
  stage: test
  script: cat compiled.txt | grep -q 'Hello world'

pack-gz:
  stage: package
  script: cat compiled.txt | gzip > packaged.gz
  needs: ["test"]
  artifacts:
    paths: [packaged.gz]

pack-iso:
  stage: package
  before_script: [apk update, apk add xorriso]
  script: mkisofs -o ./packaged.iso ./compiled.txt
  needs: ["test"]
  artifacts:
    paths: [packaged.iso]
```

Part 4: Когда запускать jobs — `rules`

Условия запуска: changes, if, exists

Pipeline Examples with rules

```yaml
stages:
  - build
  - test
  - deploy

build:
  stage: build
  script:
    - ./build_script.sh
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
    - if: '$CI_COMMIT_BRANCH == "develop"'

test:
  stage: test
  script:
    - ./test_script.sh
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
    - if: '$CI_COMMIT_BRANCH == "develop"'

deploy:
  stage: deploy
  script:
    - ./deploy_script.sh
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

Example with `rules:changes`

```yaml
build:
  stage: build
  script:
    - ./build_script.sh
  rules:
    - changes:
      - src/**

test:
  stage: test
  script:
    - ./test_script.sh
  rules:
    - changes:
      - tests/**

deploy:
  stage: deploy
  script:
    - ./deploy_script.sh
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
    - changes:
      - src/**
```

Example with `rules:if` + pipeline source

```yaml
build:
  stage: build
  script:
    - ./build_script.sh
  rules:
    - if: '$CI_PIPELINE_SOURCE == "push"'
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'

test:
  stage: test
  script:
    - ./test_script.sh
  rules:
    - if: '$CI_PIPELINE_SOURCE == "push"'
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'

deploy:
  stage: deploy
  script:
    - ./deploy_script.sh
  rules:
    - if: '$CI_PIPELINE_SOURCE == "schedule"'
```

Example with `rules:exists`

```yaml
build:
  stage: build
  script:
    - ./build_script.sh
  rules:
    - exists:
      - Dockerfile

test:
  stage: test
  script:
    - ./test_script.sh
  rules:
    - exists:
      - tests/test_script.sh

deploy:
  stage: deploy
  script:
    - ./deploy_script.sh
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
    - exists:
      - deploy/deploy_script.sh
```

Спасибо!

Теперь вы умеете писать `.gitlab-ci.yml` и управлять запуском через `rules`

Следующий шаг — includes, child pipelines и реальные паттерны
(см. полный курс GitLab CI/CD)